Autonomous Normalization and CRUD Engine for a Hybrid SQL–MongoDB Framework

Technical Report for CS 432: Databases – Assignment 2

Akash Gupta Kaushal Bule Shardul Junagade

March 22, 2026

Repository: `Kaushal845/Database_1`

Contents

1 Introduction **2**

2 Project Scope and Deliverables **2**

3 System Architecture **2**

4 Mandatory Report Questions **3**

4.1 Normalization Strategy . . . 3
4.2 Table Creation Logic (PK/FK/Indexes) . . . 3
4.3 MongoDB Design Strategy (Embed vs Reference) . . . 4
4.4 Metadata System . . . 5
4.5 CRUD Query Generation . . . 5
4.6 Performance Considerations . . . 8
4.7 Sources of Information . . . 9

5 Summary of Implemented Features **9**

6 Validation and Test Evidence **10**

7 Implementation Decisions and Trade-offs **10**

8 Known Limitations and Future Improvements **10**

9 Conclusion **11**

A Reproducibility Commands **11**

1 Introduction

This report presents the complete Assignment 2 implementation built on top of Assignment 1. The system now does more than field classification: it automatically decomposes repeating JSON entities into normalized relational tables, applies a document strategy (embed vs reference) for MongoDB, and generates CRUD execution plans from metadata. The design goal was to keep the framework flexible for semi-structured data while preserving data integrity and queryability. We therefore chose a metadata-first architecture where every major decision (placement, schema mapping, normalization, and document strategy) is persisted and reused across ingestion and query generation.

2 Project Scope and Deliverables

Compared to Assignment 1, the following capabilities were added and validated:

- Schema registration with version tracking.
- Three-path routing: SQL, MongoDB, and Buffer for undecided fields.
- SQL normalization engine for one-to-many repeating entities.
- MongoDB document strategy with automatic embed/reference decisions.
- Metadata-driven CRUD query engine (insert, read, update, delete).
- End-to-end test suite with automated validation.
- Operational improvements: cleanup command and improved observability logs.

3 System Architecture

The framework is organized as a staged pipeline:

1. **Schema Registration**: user schema is stored in a registry with version history.
2. **Ingestion**: raw JSON records are accepted and enriched with temporal fields.
3. **Metadata Interpretation**: field paths and semantic types are tracked recursively.
4. **Classification**: each field is routed to SQL, MongoDB, Buffer, or Both.
5. **Storage Strategy Generation**:
   - SQL normalizer creates child tables for repeating entities.
   - Mongo strategy decides embed/reference and target collections.
6. **Query Generation**: CRUD requests are translated into SQL/Mongo operations and merged back to JSON.

Core modules used in implementation:

- `ingestion_pipeline.py`
- `metadata_store.py`
- `placement_heuristics.py`
- `database_managers.py`

- `query_engine.py`
- `main.py` and `data_consumer.py`

4 Mandatory Report Questions

4.1 Normalization Strategy

The normalization strategy is automatic and metadata-driven. During ingestion, the pipeline now detects three SQL normalization signals:

- **Arrays of objects**: converted into child entity tables.
- **Primitive arrays**: converted into value tables (for example, `tags` → `norm_tags` with a `value` column).
- **Repeated scalar groups**: fields such as `phone1`, `phone2`, `phone3` are collapsed into one child table (for example, `norm_phone`).

Each detected entity path is converted into a normalized child table with a deterministic name:

`norm_{entity_path}`

Examples include `norm_orders`, `norm_comments`, and `norm_devices`.

Only scalar keys from object rows are materialized as child columns. For primitive arrays and scalar groups, the normalized column set is `value` plus `item_index` to preserve order.

Why this strategy works:

- Repeating groups naturally map to one-to-many structure.
- Primitive arrays and scalar groups avoid column explosion in the root table.
- Parent-child decomposition avoids oversized root rows.
- Structure evolves as new entities appear, without predefined SQL DDL files.

4.2 Table Creation Logic (PK/FK/Indexes)

The table design follows referential safety and predictable joins.

Root table:

- Table: `ingested_records`
- Primary key: `id` (auto increment)
- Join key: `sys_ingested_at` (unique)

Child tables:

- Primary key: `id`
- Foreign key: `parent_sys_ingested_at` references root `sys_ingested_at`
- Deletion behavior: `ON DELETE CASCADE`
- Ordering support: `item_index`

- Performance index: child-table index on `parent_sys_ingested_at`

This design ensures clean parent-child traversal and automatic cleanup during root deletions.

4.3 MongoDB Design Strategy (Embed vs Reference)

The Mongo strategy is entity-path based and score-driven.

For each nested field path, the pipeline computes a **reference score** S by adding points from the criteria below.

Scoring Criteria (implemented):

- **Array size**: if array length > 10, add $+2$; else if length > 3, add $+1$.
- **Array-of-objects**: if array contains object elements, add $+1$.
- **Object width**: if object key count > 8, add $+2$; else if key count > 5, add $+1$.
- **Nesting depth**: if path depth ≥ 3, add $+1$.
- **Likely shared entity**: if observed under multiple parent contexts, add $+1$.
- **Schema hint** `frequently_updated = true`: add $+1$.
- **Schema hint** `shared = true`: add $+1$.
- **Schema hint** `unbounded = true`: add $+2$.
- **Schema hint** `expected_max_items > 10`: add $+1$.

Formally, the decision rule is:

$$\text{mode}(S) = \begin{cases} \texttt{reference}, & S \geq 2 \\ \texttt{embed}, & S < 2 \end{cases}$$

This threshold keeps the strategy conservative: a single weak signal usually remains embedded, while multiple moderate signals or one strong signal can move the entity to reference mode.

Logical flow:

1. Parse nested field and compute score + reasons.
2. Choose mode (embed/reference) using threshold.
3. Persist decision telemetry in metadata.
4. Write payload to root document (embed) or a path-specific collection (reference).

Nested structural fields (`dict`/`list`) bypass Buffer warm-up and route directly to MongoDB.

Embed mode:

- Used for smaller nested objects and short arrays.
- Stored directly in root collection `ingested_records`.

Reference mode:

- Used for larger or unbounded arrays of objects.

- Used for broader nested objects likely to evolve independently.
- Used when schema hints or sharing patterns indicate independent lifecycle.
- Payloads are written to dedicated collections (for example `ref_orders`).

Reference documents store `parent_sys_ingested_at` and `entity_path` so they remain joinable and queryable from metadata.

4.4 Metadata System

The metadata layer is the intelligence center of the framework. It records both schema-level and runtime decisions.

Persisted metadata includes:

- `schema_registry`: active schema and version history.
- `fields`: appearance counts, type distributions, sampled values.
- `placement_decisions`: SQL/MongoDB/Buffer/Both with reasons.
- `buffer`: unresolved observations and resolution state.
- `field_mappings`: backend and table/collection mapping.
- `normalization`: child-table metadata by entity path.
- `mongo_strategy`: embed/reference strategy by entity path with `decision_score`, `reference_threshold`, `decision_reasons`, and `schema_hints`.

How metadata is used:

- Ingestion uses it to avoid repeated inference for finalized fields.
- Query generation uses it to resolve where every requested field lives.
- Buffer resolution uses it to migrate previously unresolved values to final backends.
- Decision telemetry supports explainability in debugging and report analysis.

4.5 CRUD Query Generation

CRUD operations are submitted as JSON and executed through a metadata-driven planner implemented in the query engine. The planner does not rely on hardcoded table or collection rules; instead, it derives execution from `field_mappings`, `normalization`, and `mongo_strategy` metadata.

Planner Stages (common to CRUD):

1. Validate request shape and operation type.
2. Build a field plan from metadata:
   - SQL root fields,
   - SQL child entities/tables,
   - Mongo root fields,
   - Mongo reference entities/collections.
3. Split filters by backend compatibility (SQL filters vs Mongo filters).
4. Execute operation-specific plan.

5. Return unified JSON response with status and operation metadata.

Insert Generation and Execution:

- `insert` accepts either one record or a list of records.
- Each record is passed through the same ingestion orchestration used by streaming ingestion.
- During ingest, metadata decides routing: SQL, MongoDB, Buffer, or Both.
- SQL writes include root table columns plus normalized child rows where needed.
- Mongo writes include embedded fields and/or referenced payload collections.
- Insert response reports success and inserted/failed counts.

Example insert request:

```
{
  "operation": "insert",
  "data": {
    "username": "crud_demo_user",
    "email": "crud_demo_user@example.com",
    "orders": [
      {
        "order_id": "ord-100",
        "item": "book",
        "quantity": 1,
        "price": 15.0
      }
    ]
  }
}
```

Read Generation and Execution (detailed):

1. Convert requested fields into a *query plan* using field mappings.
2. Ensure mandatory join keys are included (for example `username`, `sys_ingested_at`, `t_stamp`).
3. Query SQL root table first with SQL-compatible filters and selected columns.
4. Build an in-memory result map keyed by `sys_ingested_at`.
5. If SQL returns no rows, fallback to Mongo root collection for initial candidate rows.
6. Hydrate root-level Mongo fields for each key (to complete cross-backend projection).
7. For each SQL child entity in the plan, fetch child rows by `parent_sys_ingested_at` and attach as arrays.
8. For each Mongo reference entity in the plan, fetch docs by `parent_sys_ingested_at` and `entity_path`, order by `item_index`, then attach payload arrays.
9. Emit merged JSON records and include query plan details in the response.

Example read request:

```
{
  "operation": "read",
  "fields": ["username", "email", "orders", "devices"],
  "filters": {"username": "crud_demo_user"},
  "limit": 5
}
```

Update Generation and Execution:

- Update currently uses `delete_then_insert` to avoid cross-backend partial updates.
- Phase 1: execute delete plan with provided filters.
- Phase 2: execute insert plan with new payload.
- Response includes both phase outputs for observability and debugging.

Example update request:

```
{
  "operation": "update",
  "filters": {"username": "crud_demo_user"},
  "data": {
    "username": "crud_demo_user",
    "email": "crud_demo_user_updated@example.com",
    "orders": [
      {
        "order_id": "ord-101",
        "item": "phone",
        "quantity": 1,
        "price": 300.0
      }
    ]
  }
}
```

Delete Generation and Execution:

- Two delete modes are supported:
  - **Entity-level delete**: remove one mapped entity/table/collection.
  - **Root-level delete**: remove full records using filter plans for both backends.
- SQL child cleanup is automatic using `ON DELETE CASCADE`.
- Mongo root records are deleted directly; reference collections are deleted using strategy mappings and the same filter plan.
- Buffer records are also filtered and removed during root-level deletes.
- Safety guard rejects unfiltered deletes unless explicit override is provided.

Example delete requests:

```
{
  "operation": "delete",
  "filters": {"username": "crud_demo_user"}
}
```

```
{
  "operation": "delete",
  "entity": "orders",
  "filters": {"parent_sys_ingested_at": "2026-03-22T10:00:00.00000001"}
}
```

Merge and Consistency Rules:

- Merge key is `sys_ingested_at`.
- Root fields are merged first, then child/reference arrays are attached.
- Child technical columns (for example `id`, `parent_sys_ingested_at`) are removed from final JSON output.
- Result shape remains JSON-native regardless of underlying storage split.

Example query-plan fragment produced by planner:

```
{
  "sql_root_fields": ["username", "sys_ingested_at", "t_stamp"],
  "sql_child_entities": {"orders": "norm_orders"},
  "mongo_root_fields": ["username", "sys_ingested_at", "t_stamp"],
  "mongo_reference_entities": {"devices": "ref_devices"}
}
```

4.6 Performance Considerations

The implementation focuses on practical performance under evolving schemas.

- Parent-key index on child tables reduces join lookup cost.
- `ON DELETE CASCADE` removes the need for expensive manual child scans.
- Metadata-guided reads avoid querying all backends for every field.
- Reference mode reduces document rewrite overhead for large nested arrays.
- Buffer staging avoids early schema churn on low-observation fields.
- Structural nested fields skip Buffer warm-up, reducing temporary writes in `buffer_records`.
- Startup reconciliation drains historical buffered values for fields already resolved in metadata.

Extensive logging improved runtime traceability across components (consumer, pipeline, SQL manager, Mongo manager, metadata manager, and query engine), which reduces debugging time and makes backend decisions transparent during ingestion and CRUD execution.

4.7 Sources of Information

The implementation was guided by:

1. **SQLite Documentation**
   Used for understanding foreign key constraints, indexing strategies, and schema evolution.
2. **MongoDB Official Documentation**
   Referenced for embedding vs referencing design patterns and collection design practices.
3. **MongoDB Data Modeling Guide**
   Used specifically for deciding when to embed versus reference nested data.
4. **FastAPI Documentation**
   Used for API development patterns and request handling.
5. **Assignment Specification Document**

5 Summary of Implemented Features

Criterion	What was implemented and how it is evidenced
Normalization Strategy	Automatic detection of arrays-of-objects, primitive arrays, and repeated scalar groups with normalized child-table generation and parent linkage. Verified by tests that assert table creation and value ordering.
MongoDB Design	Score-based embed/reference strategy with per-entity telemetry (score, reasons, threshold, schema hints) and dedicated reference collections for high-cost nested structures.
Query Engine	Metadata-driven CRUD planner that generates SQL and Mongo operations, merges results, and supports entity-level as well as root-level deletion.
Metadata System	Persistent schema registry, field statistics, placement decisions, buffer lifecycle metadata, normalization map, and explainable Mongo strategy decision telemetry.
CRUD Functionality	Insert/read/update/delete supported through JSON interface. Update implemented with delete-then-insert consistency strategy.

6 Validation and Test Evidence

A dedicated pytest suite validates critical behavior:

- Schema registration and persistence.
- Buffer-to-final transition behavior.
- SQL normalization across arrays-of-objects, primitive arrays, and scalar groups.
- Metadata-driven CRUD lifecycle.
- Mongo strategy scoring persistence and nested-field direct Mongo routing.
- Generator endpoint and structure checks.

At the time of writing, the suite passes fully (10 tests passed), confirming stable behavior for the implemented features.

7 Implementation Decisions and Trade-offs

A few practical choices were made to keep the system robust:

- Root records are persisted to both SQL and Mongo for traceability and easier cross-backend joins.
- Buffer is treated as first-class storage, not only an in-memory state, so unresolved data is not lost.
- Update uses delete-then-insert to avoid partial divergence across two backends.

These choices prioritize correctness and debuggability over aggressive micro-optimization.

8 Known Limitations and Future Improvements

Current limitations:

- Embed/reference thresholds are still heuristic and static.
- Update strategy may be expensive for very large records due to full rewrite behavior.

Planned improvements:

- Make embed vs reference decisions smarter by learning from actual usage over time.
- Update only the changed parts of a record instead of deleting and writing the full record again.
- Improve read operations so users can request specific fields and fetch results page by page.

9 Conclusion

The Assignment 2 objective has been implemented as a working metadata-driven hybrid database framework. The system now performs autonomous normalization, document strategy selection, and CRUD query generation while preserving consistency across SQL and MongoDB.

Most importantly, the framework demonstrates the central idea of the assignment: database structure and query execution can be generated from metadata rather than hardcoded schemas, enabling flexibility for heterogeneous JSON data while retaining operational control.

A Reproducibility Commands

```
# Start generator
uvicorn main:app --reload --port 8000

# Ingest sample batches
python data_consumer.py

# View database info
python view_databases.py

# Run full tests
pytest -q

# Clean generated artifacts and project DBs
python quickstart.py clean --yes
```